

AI & Machine Learning Services

> Relevant source files

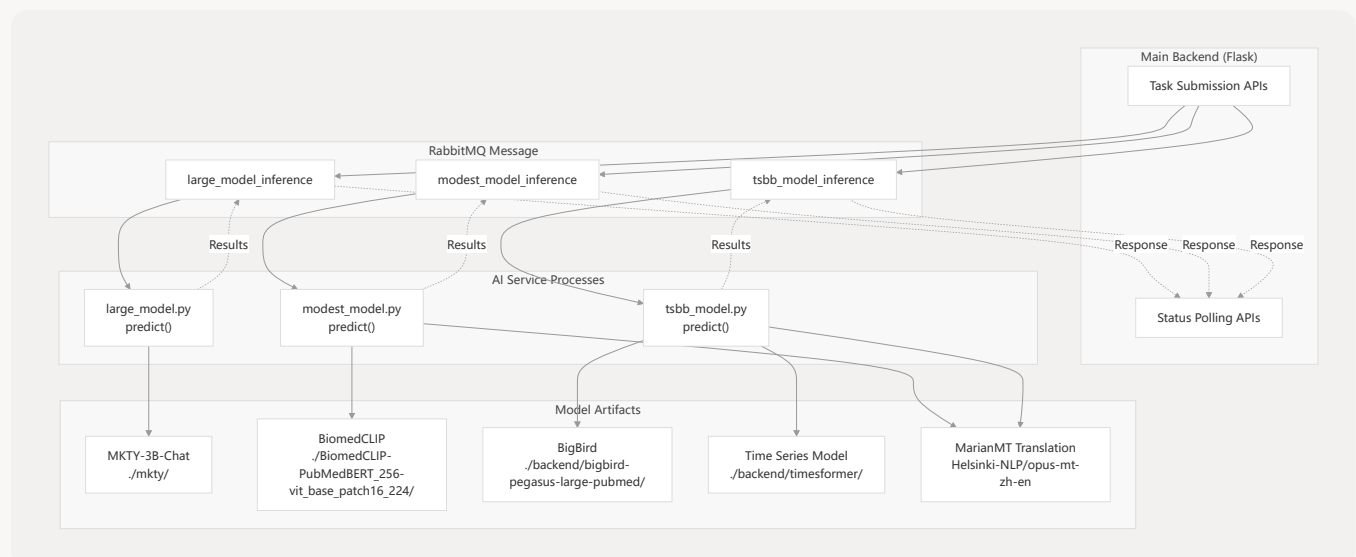
This document covers the AI and machine learning services layer of the MKTY Smart Healthcare System, which provides intelligent medical analysis capabilities through specialized model inference services. These services operate as independent message queue consumers that process computationally intensive AI tasks asynchronously from the main backend application.

The AI services layer consists of three main components: large language model inference for medical Q&A ([5.1](#)), multimodal AI services for image-text analysis ([5.2](#)), and time series prediction with text analysis ([5.3](#)). For information about the backend APIs that interface with these services, see [Core API Endpoints](#). For details about the message queue infrastructure, see [System Architecture](#).

Architecture Overview

The AI services operate as standalone Python processes that consume tasks from RabbitMQ queues and return results to the main backend. Each service is designed to handle specific types of medical AI workloads with different computational requirements.

AI Services Architecture



Sources: `backend/large_model.py` 1-63 `backend/modest_model.py` 1-81
`backend/tsbb_model.py` 1-84

Service Specifications

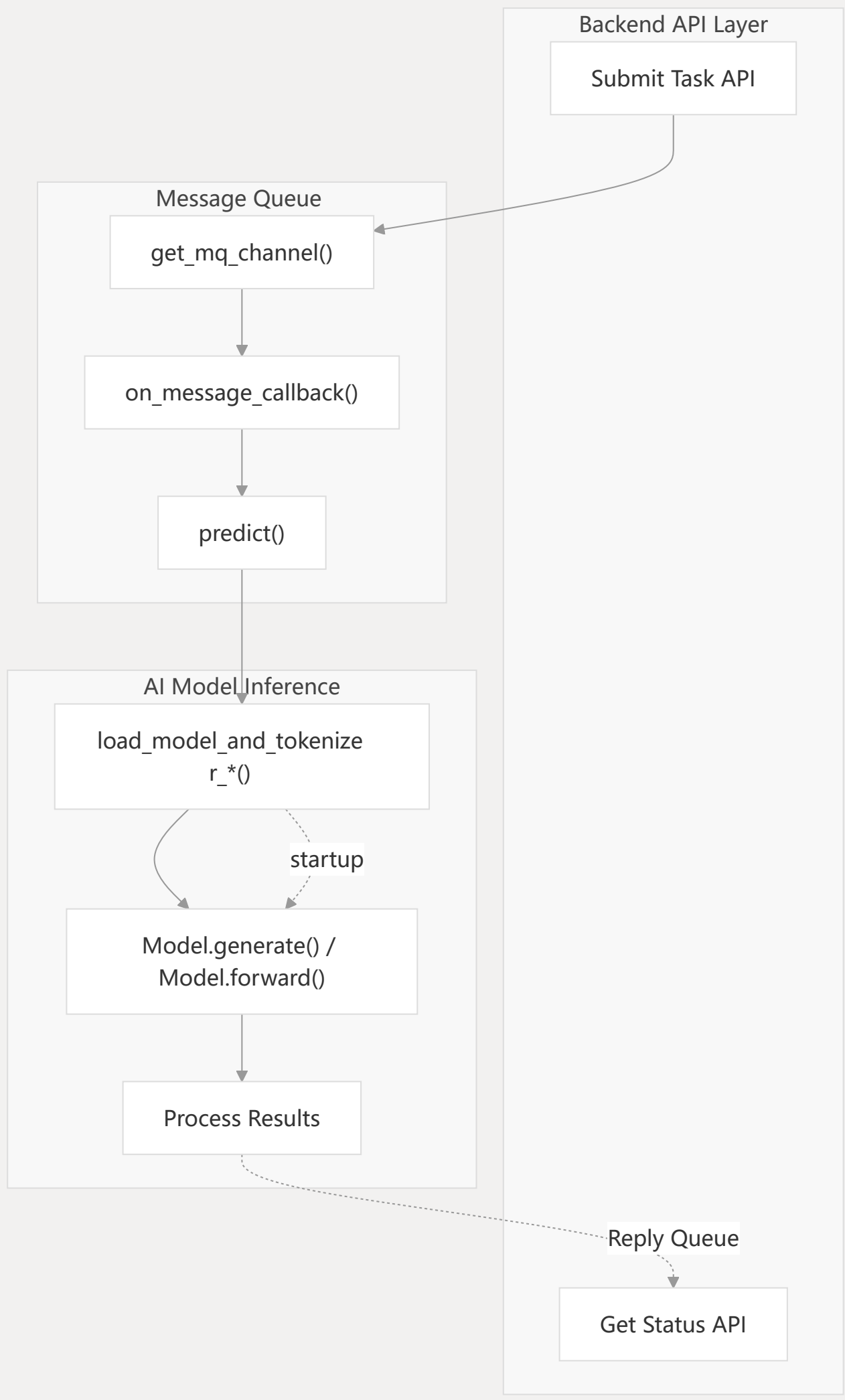
Service	Queue Name	VRAM Requirement	Primary Models	Main Function
Large Language Model	large_model_inference	8GB	MKTY-3B-Chat	Medical Q&A, discussion
Multimodal AI	modest_model_inference	2GB	BiomedCLIP, MarianMT	Image+text diagnosis
Text+Time Series	tsbb_model_inference	2GB	BigBird, GRU, MarianMT	Text features, time prediction

Sources: backend/large_model.py 16-18 backend/modest_model.py 25-27
backend/tsbb_model.py 16-18

Message Processing Pipeline

All AI services follow a common message processing pattern using RabbitMQ for asynchronous communication. Each service implements a `predict()` function that serves as the main message callback handler.

Message Flow Architecture



Sources: `backend/large_model_util.py` 66-95 `backend/modest_model_util.py` 71-100
`backend/tsbb_model_util.py` 65-94

Common Service Components

Each AI service implements the following standardized components:

- **Logging System:** `info_print()` function with timestamped log levels (info, warning, error, initial)
- **Startup Banner:** `start_print()` displays ASCII art and system information
- **MQ Connection:** `get_mq_channel()` establishes RabbitMQ connection with callback routing
- **Model Loading:** Service-specific `load_model_and_tokenizer_*` functions
- **Message Processing:** `predict()` function as the main inference callback

Sources: `backend/large_model_util.py` 20-42 `backend/large_model_util.py` 43-64
`backend/large_model_util.py` 66-95

Model Configuration and Deployment

Runtime Configuration

All services support configurable runtime parameters:

```
# Common configuration pattern across services
MQ_CONNECTION_PARAMETERS = {
    'host': 'localhost',
    'port': 5672,
    "heartbeat": 0
}
MODE = "dev" # or "prod"
VERSION = 'v1.0.0'
```

Development vs Production Mode:

- `dev` mode: Enables verbose logging of requests and responses
- `prod` mode: Minimal logging for performance

Sources: `backend/large_model.py` 11-19 `backend/modest_model.py` 20-28
`backend/tsbb_model.py` 11-20

Model Artifact Management

The system expects pre-downloaded model artifacts in specific directory structures:

Model Type	Expected Path	Download Required
MKTY-3B-Chat	<code>./mkty/</code>	Manual
BiomedCLIP	<code>./BiomedCLIP-PubMedBERT_256-vit_base_patch16_224/</code>	Manual
BigBird	<code>./backend/bigbird-pegasus-large-pubmed/</code>	Manual
MarianMT	Auto-downloaded via Hugging Face	Automatic

Important: BiomedCLIP uses OpenCLIP framework and requires manual download as it's not fully integrated with Transformers library as of the current implementation.

Sources: `backend/modest_model.py` 8-11 `.gitignore` 17-18

Hugging Face Configuration

All services configure Hugging Face endpoints for Chinese users:

```
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
```

This setting enables model downloads through the Chinese mirror for improved accessibility.

Sources: `backend/large_model_util.py` 8 `backend/modest_model_util.py` 9
`backend/tsbb_model_util.py` 8

AI & Machine Learning Services

5.1 Large Language Model Service

The large language model service provides medical Q&A capabilities using the MKTY-3B-Chat model, a specialized medical language model based on Qwen2.5-3B-Instruct. This service handles conversational medical consultations and supports both direct question-answering and multi-agent discussion mechanisms.

Model Architecture

The service implements a conversational AI system designed for medical domain expertise:

```
# MKTY-3B-Chat Model Specifications
```

```
MODEL_PATH = r"./mkty"
```

```
MAX_NEW_TOKENS = 2000
```

```
BASE_MODEL = "Qwen2.5-3B-Instruct" # Specialized for medical use
```

Hardware Requirements: 8GB VRAM minimum for optimal performance

Sources: `backend/large_model.py` 18 `backend/large_model.py` 24

Conversation Processing

The `predict()` function implements chat template processing for medical conversations:

Message Input

`message['prompt']`

`message['context']`
`List[Dict[role, content]]`

Chat Processing

`context.append({'role':
'user', 'content': prompt})`

`tokenizer.apply_chat_tem`

`tokenizer([text],
return_tensors='pt')`

Model Inference

`model.generate(**model_
inputs,`

`Extract new tokens only`

`tokenizer.batch_decode(s`

Sources: `backend/large_model.py` 26-58

Message Format

The service expects messages in the following format:

```
{
  "prompt": "Current user question",
  "context": [
    {"role": "user", "content": "Previous question"},
    {"role": "assistant", "content": "Previous response"},
    {"role": "user", "content": "Follow-up question"}
  ]
}
```

Context Management: The conversation history is maintained as a list of role-content pairs, enabling the model to provide contextually relevant medical advice based on the full conversation thread.

Sources: `backend/large_model.py` 31-41

Model Loading and Initialization

The service uses the `load_model_and_tokenizer_mkty3b()` function for model initialization:

```
def load_model_and_tokenizer_mkty3b(model_dir: str):
    model = AutoModelForCausalLM.from_pretrained(
        model_dir,
        torch_dtype="auto",
        device_map="auto"
    )
    tokenizer = AutoTokenizer.from_pretrained(model_dir)
    return model, tokenizer
```

Device Management: Uses `device_map="auto"` for automatic GPU allocation and `torch_dtype="auto"` for optimal precision selection.

Sources: `backend/large_model_util.py` 97-116

Service Lifecycle

The service follows this initialization and execution pattern:

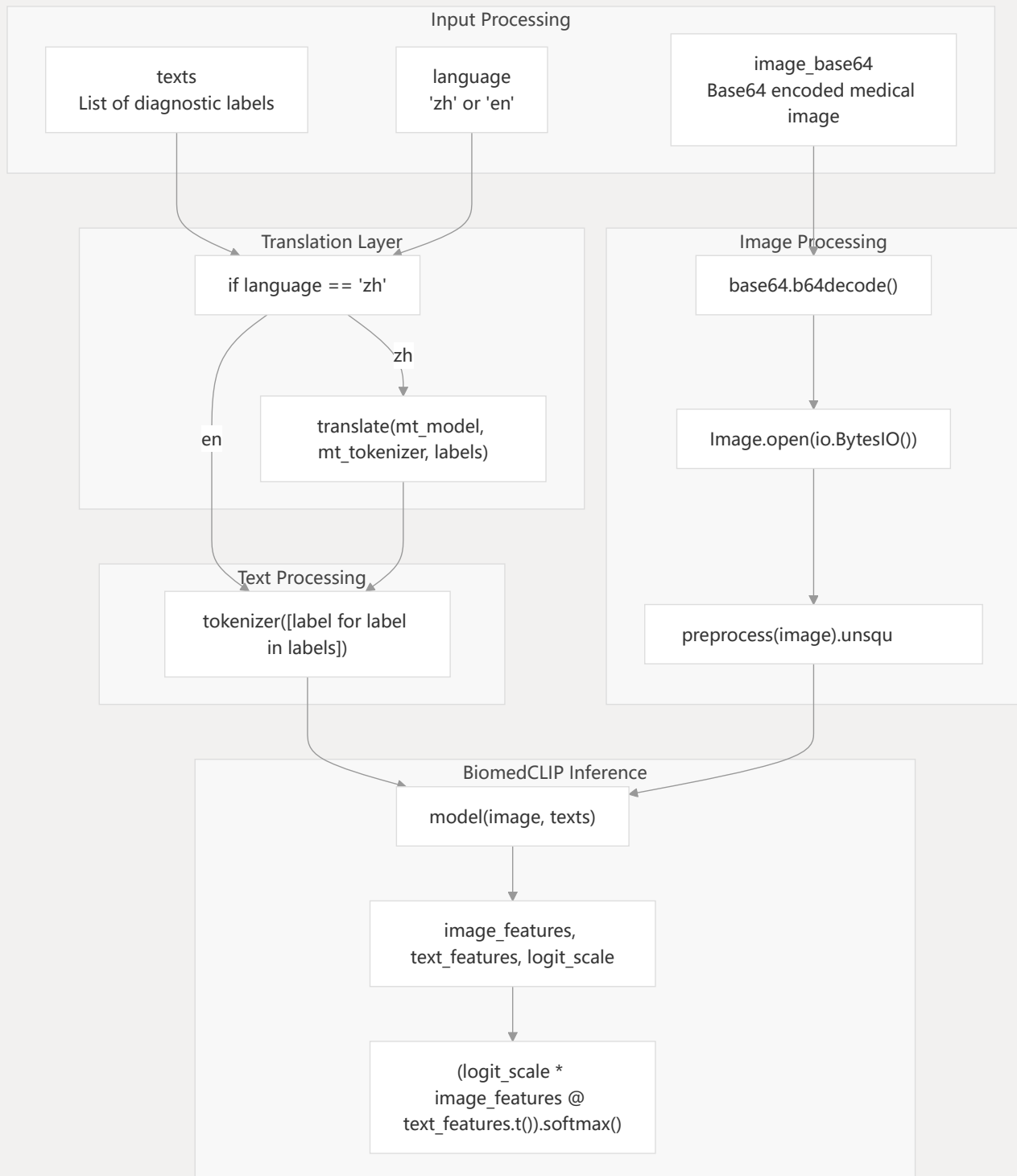
1. **Startup:** Display system banner with `start_print(VERSION)`
2. **Model Loading:** Load MKTY-3B-Chat model and tokenizer
3. **MQ Connection:** Establish RabbitMQ channel with `large_model_inference` queue
4. **Consumer Loop:** Begin consuming messages with `channel.start_consuming()`

Sources: `backend/large_model.py` 21-62

5.2 Multimodal AI Services

The multimodal AI services provide medical image analysis capabilities by combining computer vision and natural language processing. The primary components are BiomedCLIP for medical image-text understanding and MarianMT for Chinese-English translation support.

Service Architecture



Sources: `backend/modest_model.py` 36-74

BiomedCLIP Model Integration

The service uses Microsoft's BiomedCLIP model, which combines PubMedBERT for text understanding with vision transformers for medical imaging:

Model Configuration:

```
MODEL_PATH = r"./BiomedCLIP-PubMedBERT_256-vit_base_patch16_224"
model_name = "BiomedCLIP-PubMedBERT_256-vit_base_patch16_224"
```

Loading Process: The model uses OpenCLIP framework with custom configuration loading:

```
# Load configuration from open_clip_config.json
with open(os.path.join(model_dir, "open_clip_config.json"), "r") as f:
    config = json.load(f)
    model_cfg = config["model_cfg"]
    preprocess_cfg = config["preprocess_cfg"]
```

Sources: `backend/modest_model.py` 27 `backend/modest_model_util.py` 118-139

Translation Support

The service integrates MarianMT for Chinese-to-English translation to handle multilingual medical terminology:

```
def load_model_and_tokenizer_MarianMT(src: str = "zh", trg: str = "en"):
    model_name = f"Helsinki-NLP/opus-mt-{src}-{trg}"
    model = MarianMTModel.from_pretrained(model_name)
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    return model, tokenizer
```

Translation Pipeline: Text labels are automatically translated when `language` parameter is set to "zh":

```
def translate(model, tokenizer, src_text_list: str) -> list:
    result_list = []
    for text in src_text_list:
        batch = tokenizer([text], return_tensors="pt")
        generated_ids = model.generate(**batch)
        result = tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
        result_list.append(result)
    return result_list
```

Sources: `backend/modest_model_util.py` 148-164 `backend/modest_model_util.py` 166-182

Message Format and Response

Input Message Structure: